



資料結構

Data Structure

Lab 12

姓名： 陳冠豪

學號： 114310125

Lab12-Q1

Product Sorting

Code

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
using namespace std;

//用來儲存每項商品的基本資訊
struct Product {
    string name;      //商品名稱
    int expiry;       //有效期限(天數)
    int popularity;   //熱銷程度(1~10)
};

//從文字檔讀取商品資料
vector<Product> readFromFile(const string& filename) {
    vector<Product> products;
    ifstream file(filename);

    //檢查檔案是否成功開啟
    if (!file) {
        cerr << "Error opening file: " << filename << endl;
        return products;
    }

    int n;
    file >> n;      //讀取商品數量
    file.ignore();    //忽略第一行結尾的換行字元

    //逐筆讀取商品資料
    for (int i = 0; i < n; i++) {
        string line;
        getline(file, line);
```

```

//因商品名稱可能包含空白
//所以利用 rfind() 從字串尾端尋找空格位置
size_t lastSpace = line.rfind(' ');
size_t secondLastSpace = line.rfind(' ', lastSpace - 1);

try {
    //擷取商品名稱
    string name = line.substr(0, secondLastSpace);

    //擷取有效期限
    int expiry = stoi(
        line.substr(secondLastSpace + 1,
            lastSpace - secondLastSpace - 1));

    //擷取熱銷程度
    int popularity = stoi(line.substr(lastSpace + 1));

    //將商品資料存入 vector
    products.push_back({ name, expiry, popularity });
}
catch (exception& e) {
    cerr << "Invalid data format at line "
        << i + 2 << ": " << line << endl;
}
}

file.close();
return products;
}

//比較函式
//回傳 true 代表 a 商品應該排在 b 商品前面
bool compare(const Product& a, const Product& b) {

    //第一優先條件：
    //有效期限越短，越需要優先處理
    if (a.expiry != b.expiry)
        return a.expiry < b.expiry;
}

```

```

//第二優先條件：
//若有效期限相同，則依照熱銷程度排序
//熱銷程度越高排越前面
return a.popularity > b.popularity;
}

//Quick Sort 排序演算法
void quickSort(vector<Product>& products, int left, int right) {

    //當區間只剩一個元素或沒有元素時停止遞迴
    if (left >= right)
        return;

    //選擇中間元素作為 Pivot
    Product pivot = products[(left + right) / 2];

    int i = left;
    int j = right;

    //分割區間(Partition)
    while (i <= j) {

        //從左邊尋找不符合排序規則的元素
        while (compare(products[i], pivot))
            i++;

        //從右邊尋找不符合排序規則的元素
        while (compare(pivot, products[j]))
            j--;

        //若找到兩個位置需要交換
        if (i <= j) {
            swap(products[i], products[j]);
            i++;
            j--;
        }
    }
}

```

```

//遞迴處理 Pivot 左側資料
quickSort(products, left, j);

//遞迴處理 Pivot 右側資料
quickSort(products, i, right);
}

int main() {

//指定輸入檔案路徑
string filename = "C:/input/input3 (2).txt";

//讀取商品資料
vector<Product> products = readFromFile(filename);

//若沒有成功讀取資料則結束程式
if (products.empty()) {
    cerr << "No valid data found in file." << endl;
    return -1;
}

//顯示原始資料
cout << "Input Products:" << endl;
for (const Product& p : products) {
    cout << p.name << " "
        << p.expiry << " "
        << p.popularity << endl;
}

cout << endl;

//呼叫 Quick Sort 進行排序
quickSort(products, 0, products.size() - 1);

//顯示排序後結果
cout << "商品排序:" << endl;
for (const Product& p : products) {

```

```

        cout << p.name << endl;
    }

    system("pause");
    return 0;
}

```

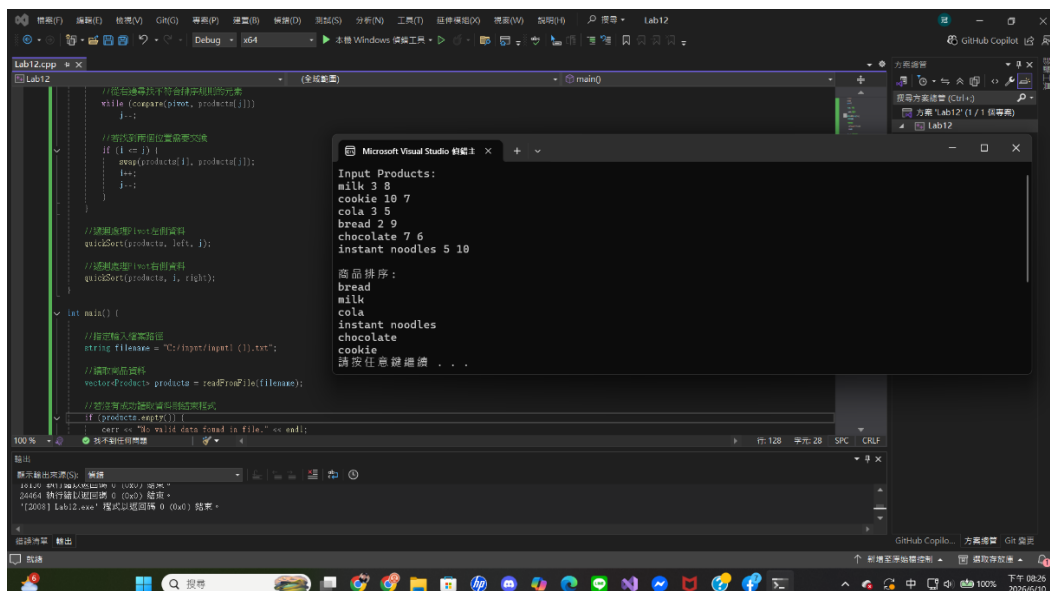
Result

```

Microsoft Visual Studio 偵錯主
Input Products:
milk 3 8
cookie 10 7
cola 3 5
bread 2 9
chocolate 7 6
instant noodles 5 10

商品排序:
bread
milk
cola
instant noodles
chocolate
cookie
請按任意鍵繼續 . . .

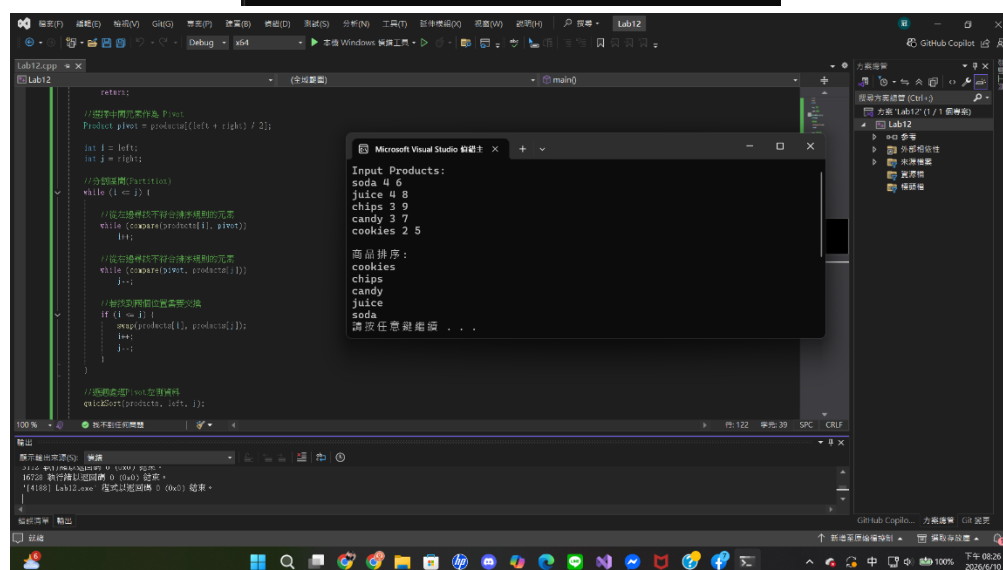
```



```
C:\Users\user\source\repos\La X + v

Input Products:
soda 4 6
juice 4 8
chips 3 9
candy 3 7
cookies 2 5

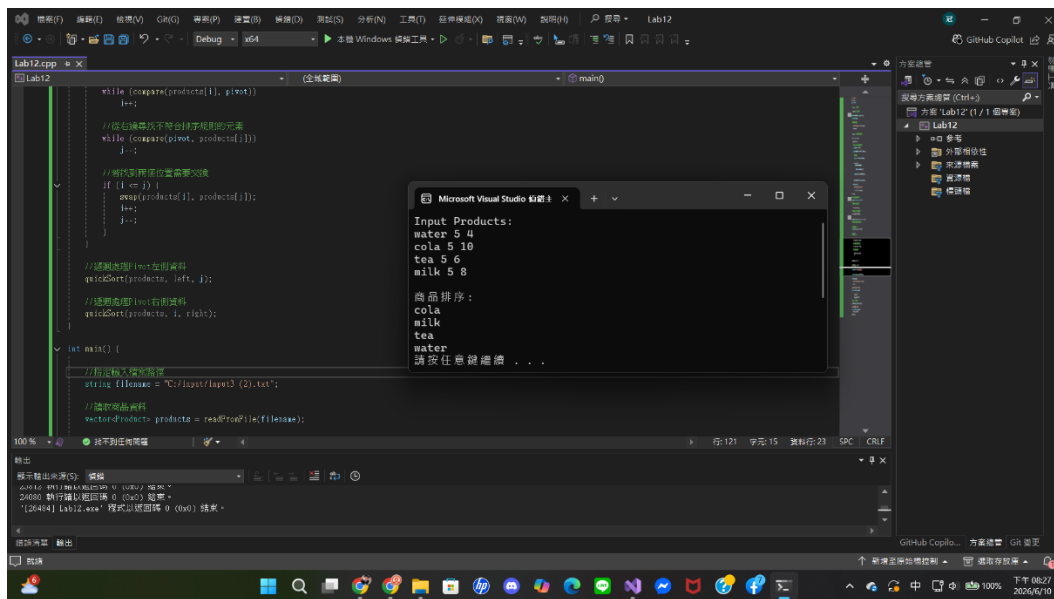
商品排序:
cookies
chips
candy
juice
soda
請按任意鍵繼續 . . . |
```



```
Microsoft Visual Studio 偵錯主 X + v

Input Products:
water 5 4
cola 5 10
tea 5 6
milk 5 8

商品排序:
cola
milk
tea
water
請按任意鍵繼續 . . .
```



```
while (compare(products[i], pivot))  
    i++;  
  
//將比基準值下等會排字順序的元素  
while (compare(pivot, products[j]))  
    j--;  
  
//倘若到陣首位置需要交換  
if (i <= j) {  
    swap(products[i], products[j]);  
    i++;  
    j--;  
}  
  
//遞迴處理 i 左側資料  
quickSort(products, left, i);  
  
//遞迴處理 i 右側資料  
quickSort(products, i, right);  
  
int main() {  
    //指定輸入檔案位置  
    string filename = "C:/input/layer3 (2).txt";  
  
    //讀取商品資料  
    vector<products> products = readFromFile(filename);  
  
    //顯示輸出來源(S) 編輯  
    cout << endl << "Input Products:" << endl << "cola 5 10" << endl << "tea 5 6" << endl << "milk 5 8" << endl << "商品排序:" << endl << "cola" << endl << "milk" << endl << "tea" << endl << "water" << endl << "請按任意鍵繼續..." << endl;
```

Discussion

為什麼選擇此演算法：

本程式選擇使用 Quick Sort 作為商品排序的演算法，主要是因為它在處理大量資料時具有不錯的效率，而且實作方式相對容易理解。由於本題需要根據商品的有效期限以及熱銷程度進行排序，因此透過自訂比較函式即可靈活地定義排序規則，再搭配 Quick Sort 的分而治之（Divide and Conquer）概念，能快速將資料依照優先順序排列完成。此外，本程式選擇中間元素作為 Pivot（基準值），可以降低資料分割不均的情況發生，使整體排序效能更加穩定，因此非常適合作為本次商品管理系統的排序方法。

時間複雜度分析：

在時間複雜度方面，Quick Sort 的平均時間複雜度為 $O(n \log n)$ ，表示即使商品數量增加，排序所需的時間也能維持在相對有效率的範圍內。雖然在極端情況下，例如每次選到的 Pivot 都造成資料分割非常不平均時，時間複雜度可能退化為 $O(n^2)$ ，但由於本程式採用中間元素作為 Pivot，因此能有效降低發生最差情況的機率。綜合來看，Quick Sort 在大多數實際應用情境下都能提供良好的排序效率，因此能夠有效完成本系統的商品優先排序需求。